

NAME- Shiv Suryawanshi

PRN NO- 202501050033

BATCH- CH2

PRACTICAL NO.2

Write a Python program that implements a menu-driven interface for managing a list of integers. The program should have the following menu options:

1. Add
2. Remove
3. Display
4. Quit

The program should repeatedly prompt the user to enter a choice from the menu. Depending on the choice selected, the program should perform the following actions:

- **Add:** Prompts the user to enter an integer and add it to the integer list. If the input is not a valid integer, display "Invalid input".
- **Remove:** Prompts the user to enter an integer to remove from the list. If the integer is found in the list, remove it; otherwise, display "Element not found". If the list is empty, display "List is empty".
- **Display:** Displays the current list of integers. If the list is empty, display "List is empty".
- **Quit:** Exits the program.
- The program should handle invalid menu choices by displaying "Invalid choice". Ensure that the program continues to prompt the user until they choose to quit (option 4).

CODE:

```
lst = []
```

```
while True:
```

```
    print("1. Add")
```

```
print("2. Remove")    print("3.  
Display")    print("4. Quit")
```

```
choice = int(input("Enter choice: "))
```

```
if choice == 1:        num =  
int(input("Integer: "))  
lst.append(num)        print("List after  
adding:", lst)
```

```
elif choice == 2:      if  
len(lst) == 0:  
    print("List is empty")    else:  
    num = int(input("Integer: "))  
if num in lst:          lst.remove(num)  
print("List after removing:", lst)  
else:                   print("Element not  
found")
```

```
elif choice == 3:  
if len(lst) == 0:  
    print("List is empty")    else:  
    print(lst)
```

```
elif choice == 4:
```

break

```
else:    print("Invalid  
choice")
```

OUTPUT:



```
1. Add  
2. Remove  
3. Display  
4. Quit  
Enter choice: 1  
Integer: 11  
List after adding: [11]  
1. Add  
2. Remove  
3. Display  
4. Quit  
Enter choice: 1  
Integer: 25  
List after adding: [11, 25]  
1. Add  
2. Remove  
3. Display
```

Write a Python program to perform insertion, update, deletion, and traversal operations on a dictionary. An initial dictionary containing 10 predefined records is already given in the program.

Operations to be Performed:

1. Insertion – Insert a new key-value pair into the dictionary using user input.
2. Update – Update the value of an existing key using user input.
3. Deletion – Delete a specified key from the dictionary using user input. 4.
- Traversal – Traverse the final dictionary and display all key-value pairs.

Input and Output Format:

1. Read an integer representing the key to be inserted and a string representing its value. Insert this new key-value pair into the dictionary and display the dictionary after insertion.
2. Read an integer representing the key to be updated and a string representing the new value. Update the value of the specified key (only if it exists) and display the dictionary after the update.
3. Read an integer representing the key to be deleted. Delete the specified key from the dictionary (only if it exists) and display the dictionary after deletion.
4. Finally, traverse the dictionary and display all key-value pairs.

The program should also display the original dictionary before performing any operations. Each output should be printed with appropriate messages.

CODE:

```
# Initial dictionary with 10 predefined records student
```

```
= {
```

```
    1: "Amit",
```

```
    2: "Riya",
```

```
    3: "Kiran",
```

```
    4: "Neha",
```

```
5: "Arjun",  
6: "Pooja",  
7: "Rahul",  
8: "Sneha",  
9: "Vikram",  
10: "Anjali"  
}
```

```
print("Original Dictionary:", student)
```

```
# Insertion k1 = int(input()) v1 =  
input() student[k1] = v1  
print("After Insertion:", student)
```

```
# Update k2 = int(input()) v2 =  
input() if k2 in student:  
student[k2] = v2 print("After  
Update:", student)
```

```
# Deletion k3 =  
int(input()) if k3 in  
student: del  
student[k3]  
print("After Deletion:", student)
```

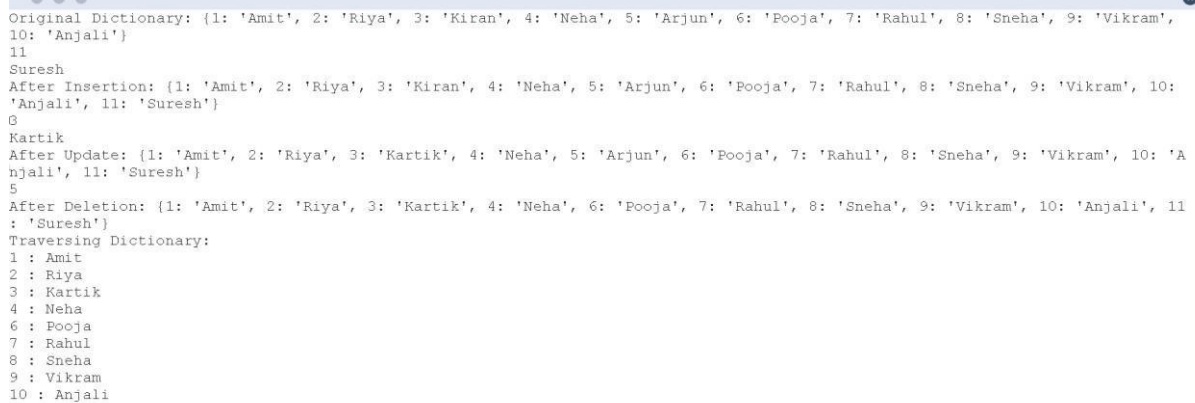
```
# Traversal
```

```
print("Traversing Dictionary:") for
```

```
key, value in student.items():
```

```
print(key, ":", value)
```

OUTPUT:



```
Original Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali'}
11
Suresh
After Insertion: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}
0
Kartik
After Update: {1: 'Amit', 2: 'Riya', 3: 'Kartik', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}
5
After Deletion: {1: 'Amit', 2: 'Riya', 3: 'Kartik', 4: 'Neha', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}
Traversing Dictionary:
1 : Amit
2 : Riya
3 : Kartik
4 : Neha
6 : Pooja
7 : Rahul
8 : Sneha
9 : Vikram
10 : Anjali
```

Write a program to check whether the given element is present or not in the array of elements using linear search.

Input format:

- The first line of input contains the array of integers which are separated by space
- The last line of input contains the key element to be searched

Output format:

- If the element is found, print the index. If the element found multiple times, print the starting index

- If the element is not found, print **Not found**.

Sample Test Case:

Input:

1 2 3 4 3 5 6

3

Output:

2

CODE:

```
arr = list(map(int, input().split())) key
```

```
= int(input())
```

```
found = False
```

```
for i in range(len(arr)): if
```

```
arr[i] == key:
```

```
print(i) found =
```

```
True break
```

```
if not found:
```

```
    print("Not found")
```

OUTPUT:



```
$ python CTF17092.py
1 2 3 4 3 5 6
3
2
=== YOUR PROGRAM HAS ENDED ===
```

You are provided with the heights of 11 cricket players (in centimeters). Your task is to identify the tallest player, who will be selected as the captain of the team.

Input Format:

The first line of input will contain 11 integers, each representing the height of a player (in centimeters), each separated by a space.

Output Format

The output should be the height (in centimeters) of the tallest player.

CODE:

```
heights = list(map(int, input().split())) print(max(heights))
```

OUTPUT:

